SQL Benchmark Evaluation Strategy

Overview: This document outlines the evaluation strategy and multi-tiered benchmarking framework designed to assess the performance of the SQL generation agent against the dataset. The strategy ensures rigorous tracking of functional correctness, structural optimization, pipeline integrity, and runtime resilience.

Core Evaluation Metrics: The core performance of the agent will be automatically graded and measured using the following foundational metrics:

1. Execution Accuracy (EX): Measures whether the generated SQL query yields the exact same functional output and dataset as the ground truth query when executed on the target database environment.

2. Exact Set Match (ESM): Evaluates the structural query equivalence. It determines whether the abstract syntax tree (AST) and core component blocks of the generated query align logically with the standard ground truth solution, irrespective of minor formatting variations.

3. Valid SQL Execution Rate (VSER): Evaluates pipeline integrity by measuring the percentage of agent-generated queries that execute successfully without throwing syntax, database schema, or runtime errors.

4. Schema Selection Precision (SSP): Gauges contextual mapping capabilities by assessing how accurately the agent identifies and selects the correct tables, keys, and specific columns from the database schema required to address the given prompt.

Advanced Framework Capabilities: The benchmarking pipeline incorporates several system-level testing methodologies to guarantee robust, real-world deployment viability:

1. Semantic Equivalence Validation: The evaluation environment verifies that alternative query structures providing identical business answers are correctly credited, eliminating penalties for varied semantic writing styles.

2. Self-Correction Loops & Resilience: The agent is evaluated on its capacity to process live PostgreSQL error logs and error feedback to iteratively debug, re-factor, and fix its initial query structure autonomously within defined token bounds.

3. Computational Efficiency & Optimization: Queries are tracked for structural optimization, resource utilization, and execution cost to ensure the generated code is optimized for processing efficiency on large-scale relational datasets.